3.1.2

```
(define let->application
  (lambda (exp)
    (cons (list 'lambda (let-vars exp) (let-body exp))
          (let-exps exp))))

(define let-vars
  (lambda (exp)
    (map car (cadr exp))))
(define let-exps
  (lambda (exp)
    (map cadr (cadr exp))))
(define let-body
  (lambda (exp)
    (caddr exp)))
```

3.1.3

```
(letrec ((subst
          (lambda (new old slst)
            (if (null? slst)
                '()
                (if (symbol? (car slst))
                    (if (eq? (car slst) old)
                        (cons new (subst new old (cdr slst)))
                        (cons (car slst)
                              (subst new old (cdr slst))))
                    (cons (subst new old (car slst))
                          (subst new old (cdr slst)))))))))
  (subst 'a 'b '(a b c))) ; letrec needs a body
```

3.1.4 (extends 2.3.10, 3.1.2)

Since `let` and `letrec` have equivalent selectors, we condense both cases under one clause.

```
(define free-vars
  (lambda (exp)
    (no-duplicates (all-free exp '()))))
(define all-free
  (lambda (exp params)
    (if (varref? exp)
        (if (member exp params)
            '()
            (list exp))
        (if (if-exp? exp)
            (append (all-free (predicate exp) params)
                    (all-free (consequent exp) params)
                    (all-free (alternative exp) params))
            (if (or (let-exp? exp) (letrec-exp? exp))
                (append (reduce append '()
                                (map (lambda (l)
                                       (all-free l params))
                                     (let-exps exp)))
                        (all-free (let-body exp)
                                  (append (let-vars exp)
                                          params)))
                (if (lambda-exp? exp)
                    (all-free (lambda-body exp)
                              (append (formal-parameters exp)
                                      params))
                    (append (all-free (operator exp) params)
                            (reduce append
                                    '()
                                    (map (lambda (l)
                                           (all-free l params))
                                         (operands exp)))))))))))
```

```
(define bound-vars
  (lambda (exp)
    (no-duplicates (all-bound exp '()))))
(define all-bound
  (lambda (exp params)
    (if (varref? exp)
        '()
        (if (if-exp? exp)
            (append (all-bound (predicate exp) params)
                    (all-bound (consequent exp) params)
                    (all-bound (alternative exp) params))
            (if (or (let-exp? exp) (letrec-exp? exp))
                (append (occurrences (let-vars exp)
                                     (free-vars (let-body exp)))
                        (reduce append '()
                                (map (lambda (l)
                                       (all-bound l params))
                                     (let-exps exp)))
                        (all-bound (let-body exp)
                                   (append (let-vars exp)
                                           params)))
                (if (lambda-exp? exp)
                    (append (occurrences
                             (formal-parameters exp)
                             (free-vars (lambda-body exp)))
                            (all-bound (lambda-body exp)
                                       (append
                                        (formal-parameters exp)
                                        params)))
                    (append (all-bound (operator exp) params)
                            (reduce append
                                    '()
```

```
                                        (map (lambda (l)
                                               (all-bound l params))
                                             (operands exp)))))))))))

(define let-exp?
  (lambda (exp)
    (eq? 'let (car exp))))
(define let-body caddr)
(define letrec-exp? ; not used
  (lambda (exp)
    (eq? 'letrec (car exp))))
(define letrec-body caddr) ; not used

(define lambda-body caddr)
```

3.1.5 (extends 2.3.10)

```
(define transform-exp
  (lambda (exp env free)
    (if (varref? exp)
        (list exp ': (depth exp env) (position exp env free))
        (if (if-exp? exp)
            (list 'if
                  (transform-exp (predicate exp) env free)
                  (transform-exp (consequent exp) env free)
                  (transform-exp (alternative exp) env free))
            (if (let-exp? exp)
                (list 'let
                      (map (lambda (pair)
                             (list
                              (car pair)
                              (transform-exp
                               (cadr pair) env free)))
                           (let-bindings exp))
```

```
                    (transform-exp
                     (let-body exp)
                     (extend (let-vars exp) env)
                     free))
                (if (letrec-exp? exp)
                    (list 'letrec
                          (map (lambda (pair)
                                 (list
                                  (car pair)
                                  (transform-exp
                                   (cadr pair) env free)))
                               (letrec-bindings exp))
                          (transform-exp
                           (letrec-body exp)
                           (extend (letrec-vars exp) env)
                           free))
                    (if (lambda-exp? exp)
                        (list 'lambda
                              (formal-parameters exp)
                              (transform-exp
                               (lambda-body exp)
                               (extend (formal-parameters exp)
                                       env)
                               free))
                        (map (lambda (l)
                               (transform-exp l env free))
                             (extend (operator exp)
                                     (operands exp)))))))))

(define let-bindings cadr)
(define letrec-bindings cadr)
(define letrec-vars
```

```
  (lambda (exp)
    (map car (cadr exp))))
(define letrec-exps
  (lambda (exp)
    (map cadr (cadr exp))))
(define letrec-body
  (lambda (exp)
    (caddr exp)))
```

3.2.1

```
(define and-proc
  (lambda tests
    (letrec ((and-helper
              (lambda (tests)
                (if (null? (cdr tests))
                    (car tests)
                    (if (car tests)
                        (and-helper (cdr tests))
                        #f)))))
      (and-helper tests))))

(define or-proc
  (lambda tests
    (letrec ((or-helper
              (lambda (tests)
                (if (null? (cdr tests))
                    (car tests)
                    (let ((*value* (car tests)))
                      (if *value*
                          *value*
                          (or-helper (cdr tests))))))))
      (or-helper tests))))
```

3.3.1

```
(define if->cond
  (lambda (exp)
    (letrec ((i->c
              (lambda (exp)
                (if (if-exp? (alternative exp))
                    (cons (list (predicate exp) (consequent exp))
                          (i->c (alternative exp)))
                    (cons (list (predicate exp) (consequent exp))
                          (list (list 'else (alternative exp))))))))
      (cons 'cond (i->c exp)))))

(define if-exp?
  (lambda (exp)
    (and (pair? exp) (= (length exp) 4) (eq? (car exp) 'if))))

(define cond->if
  (lambda (exp)
    (letrec ((c->i
              (lambda (exp)
                (if (else-clause? (next-clause exp))
                    (list 'if
                          (clause-test (first-clause exp))
                          (clause-consequent (first-clause exp))
                          (clause-consequent (next-clause exp)))
                    (list 'if
                          (clause-test (first-clause exp))
                          (clause-consequent (first-clause exp))
                          (c->i (rest-clauses exp)))))))
      (c->i (clauses exp)))))

(define first-clause car)
(define clause-test car)
```

```
(define clause-consequent cadr)
(define rest-clauses cdr)
(define next-clause cadr)
(define else-clause? (lambda (clause) (eq? (car clause) 'else)))
(define clauses cdr)
```

3.3.2 (extends 2.3.10, 3.1.5, 3.3.1)

```
(define lexical-address
  (lambda (exp)
    (transform-exp exp '() (free-vars exp))))

(define all-free
  (lambda (exp params)
    (if (varref? exp)
        (if (member exp params)
            '()
            (list exp))
        (if (if-exp? exp)
            (append (all-free (predicate exp) params)
                    (all-free (consequent exp) params)
                    (all-free (alternative exp) params))
            (if (cond-exp? exp)
                (reduce
                 append '()
                 (map (lambda (clause)
                        (if (else-clause? clause)
                            (all-free
                             (clause-consequent clause)
                             params)
                            (append
                             (all-free
                              (clause-test clause) params)
                             (all-free
```

```
                (clause-consequent clause) params)))
      (clauses exp)))
(if (let-exp? exp)
    (append
     (reduce append '()
             (map (lambda (l) (all-free l params))
                  (let-exps exp)))
     (all-free (let-body exp)
               (append (let-vars exp) params)))
    (if (letrec-exp? exp)
        (append
         (reduce append '()
                 (map (lambda (l)
                        (all-free l params))
                      (letrec-exps exp)))
         (all-free (letrec-body exp)
                   (append (letrec-vars exp)
                           params)))
        (if (lambda-exp? exp)
            (all-free (lambda-body exp)
                      (append
                       (formal-parameters exp)
                       params))
            (append (all-free (operator exp)
                              params)
                    (reduce
                     append
                     '()
                     (map (lambda (l)
                            (all-free l params))
                          (operands exp)))))))))))
```

```
(define transform-exp
  (lambda (exp env free)
    (if (varref? exp)
        (list exp ': (depth exp env) (position exp env free))
        (if (if-exp? exp)
            (list 'if
                  (transform-exp (predicate exp) env free)
                  (transform-exp (consequent exp) env free)
                  (transform-exp (alternative exp) env free))
            (if (cond-exp? exp)
                (cons
                 'cond
                 (map (lambda (clause)
                        (if (else-clause? clause)
                            (list
                             'else
                             (transform-exp
                              (clause-consequent clause)
                              env
                              free))
                            (list
                             (transform-exp
                              (clause-test clause) env free)
                             (transform-exp
                              (clause-consequent clause)
                              env
                              free))))
                      (clauses exp)))
                (if (let-exp? exp)
                    (list
                     'let
                     (map (lambda (pair)
```

```
          (list
           (car pair)
           (transform-exp (cadr pair) env free))
         (let-bindings exp))
  (transform-exp
   (let-body exp)
   (extend (let-vars exp) env) free))
 (if (letrec-exp? exp)
     (list
      'letrec
      (map (lambda (pair)
             (list
              (car pair)
              (transform-exp
               (cadr pair) env free)))
           (letrec-bindings exp))
      (transform-exp
       (letrec-body exp)
       (extend (letrec-vars exp) env)
       free))
     (if (lambda-exp? exp)
         (list
          'lambda
          (formal-parameters exp)
          (transform-exp
           (lambda-body exp)
           (extend
            (formal-parameters exp) env)
           free))
         (map (lambda (l)
                (transform-exp l env free))
              (extend
```

```
                                        (operator exp)
                                        (operands exp)))))))))))
```

```
(define cond-exp?
  (lambda (exp)
    (and (pair? exp) (eq? 'cond (car exp)))))
```

3.3.3

```
(let ((*key* (get-position-of item)))
  (cond ((memv *key* '(one first 1)) (car lst))
        ((memv *key* '(two second 2)) (cadr lst))
        (else (error "Invalid position"))))
```

3.3.4

```
(case color
  ((red pink mauve) 'reddish)
  ((white off-white cream) 'whiteish)
  ((blue azure navy) 'blueish)
  (else (error "Invalid color" color)))
```

The rest of the exercises are similar to the third edition of the book.